

AWS re:Invent

NOV. 28 – DEC. 2, 2022 | LAS VEGAS, NV

CON301

Well-built and secure Kubernetes manifests with cdk8s

Eli Polonsky (He/Him)

Sr. Software Development Engineer
AWS

David Feldstein (He/Him)

Sr. Containers Specialist
AWS



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Agenda

Problem statement

cdk8s

cdk8s-plus

Demo

Protecting your manifests

Demo

Resources



Problem statement

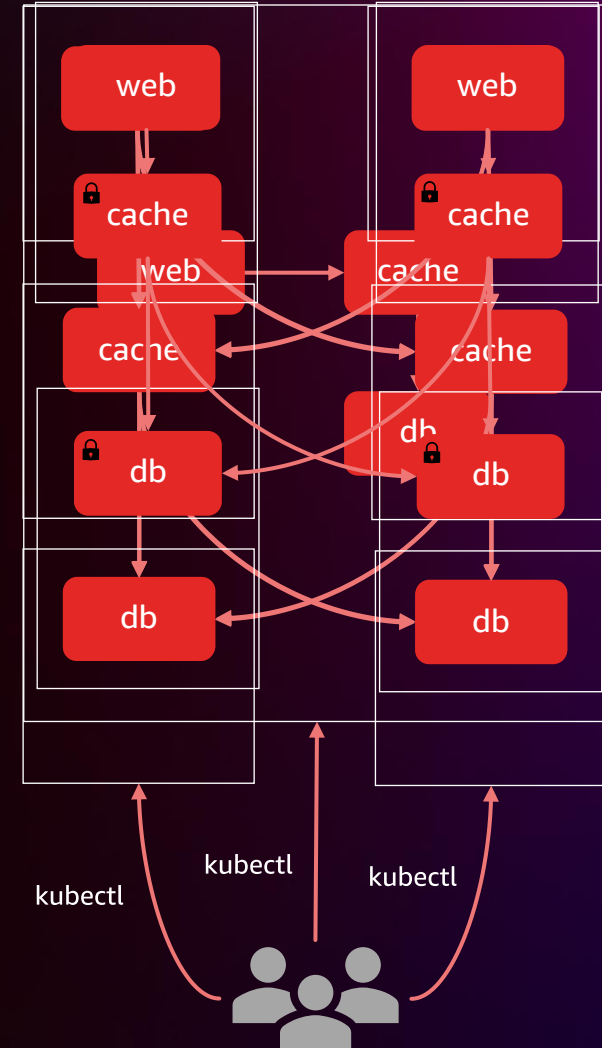
Authoring production-grade Kubernetes manifests is a complex task.

Or rather, more complex than it could be.

Problem statement

Production requirements:

- Everything needs to be highly available.
- Web and cache need to be collocated with each other.
- Web is only allowed to connect with the cache.
- Cache only allows web connections.
- Cache is only allowed to connect to the db.
- db only allows cache connections.
- Grant the “frontoffice” group permissions to manage this application.



Problem statement

Kubernetes Documentation / Concepts / Scheduling, Preemption and

Network Policies

Assigning Pods to Nodes

You can constrain a Pod so that it is restricted to run on particular nodes. There are several ways to do this and the recommended approach is to use the `nodeSelector` field. Often, you do not need to set any such constraints; the Kubernetes scheduler will place Pods on nodes that have sufficient free resources. However, there are some circumstances where you may want to constrain a Pod's placement, for example, to ensure that a Pod ends up on a node that has sufficient free resources. However, there are some circumstances where you may want to constrain a Pod's placement, for example, to ensure that a Pod ends up on a node that has sufficient free resources.

the Pod deploys to, for example, to ensure that a Pod ends up on a node that has sufficient free resources. However, there are some circumstances where you may want to constrain a Pod's placement, for example, to ensure that a Pod ends up on a node that has sufficient free resources.

Advanced Scheduling and Preemption

Authorize actions in clusters using role-based access control

Overview

This page shows you how to authorize actions on resources in your Google Kubernetes Engine (GKE) clusters using the built-in role-based access control (RBAC) mechanism in Kubernetes.

For example, using affinity rules, you could spread other services. Anti-affinity rules allow you to prevent the same nodes as pods of another service that are known pods of the first service. Or, you could spread the pods reduce correlated failures.

Using RBAC Authorization

Role-based access control (RBAC) is a method for controlling the roles of individual users within your organization.

RBAC authorization uses the `rbac.authorization.k8s.io` API to dynamically configure policies through the `API server`.

To enable RBAC, start the API server with the `--enable-rbac` flag. For example:

```
kube-apiserver --enable-rbac
```

Configure network policies for applications

This tutorial demonstrates how to use cluster network policies to control which Pods receive incoming network traffic, and which Pods can send outgoing traffic. For more information, see [Creating a cluster network policy](#).

Network policies allow you to limit connections between Pods. Therefore, using network policies provide better security by reducing the compromise radius.

Note that the network policies determine whether a connection is allowed, and they do not offer higher level features like authorization or secure transport (like SSL/TLS).

Objectives

In this tutorial, you will learn:

- How to create clusters with Network Policy enforcement
- How to restrict incoming traffic to Pods using labels
- How to restrict outgoing traffic from Pods using labels

```
apiVersion: v1
kind: Service
metadata:
  name: aws-container-day-web-service-c831a3cb
spec:
  type: LoadBalancer
  selector:
    cdk8s.io/metadata.addr: aws-container-day-web-c80b2a40
  ports:
    - port: 9090
      targetPort: 9090
  externalIPs: []

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: rolec8bcfdafc6bfa0f3a69e139bdaa3f2809429fb09b-c87dadbd0
rules:
  - apiGroups:
    - apps
    resourceNames:
    - aws-container-day-web-c8024530
    resources:
    - deployments
    verbs:
    - get
    - list
    - watch
    - create
    - update
    - patch
    - delete
    - deletecollection

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: rolebindingc8bcfdafc6bfa0f3a69e139bdaa3f28094-c85c218e
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: rolec8bcfdafc6bfa0f3a69e139bdaa3f2809429fb09b-c87dadbd0
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: frontoffice
```

430 lines
22 resources
50+ properties

CDK For Kubernetes (CDK8s)

An open-source multi-language software development framework for modeling Kubernetes resources as reusable components



Go from code to config

Define Kubernetes applications and architectures using familiar programming languages



Cut copy & paste

Turn best practices into code libraries and share easily

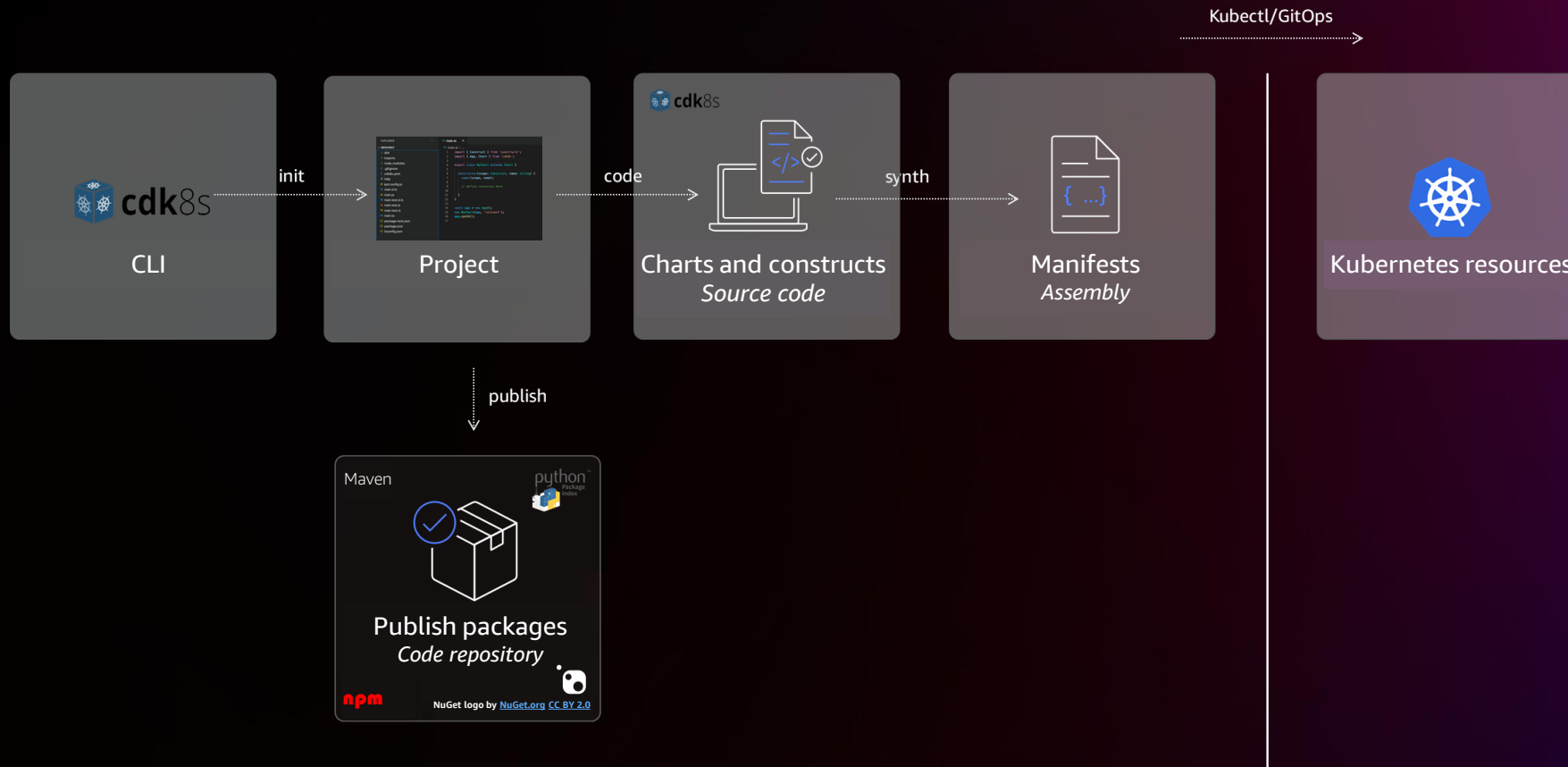


Run everywhere

CDK8s runs locally and generates YAML you can deploy to any cluster, anywhere



cdk8s: Workflow



cdk8s: In a nutshell

```
new k8s.KubeDeployment(this, 'Nginx', {
  metadata: {
    name: 'nginx'
  },
  spec: {
    template: {
      metadata: {
        labels: { app: 'nginx' }
      },
      spec: {
        containers: [{
          name: 'nginx-main',
          image: 'nginx'
        }]
      }
    },
    selector: {
      matchLabels: { app: 'nginx' }
    }
  }
});
```

cdk8s synth

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - image: nginx
          name: nginx-main
```

“Perfection is achieved, not when there is nothing more to add, but when there is nothing left to take away.”

Antoine de Saint-Exupery

Writer, poet, aristocrat, journalist, and pioneer aviator

cdk8s-plus: An intent-based API abstraction

State

```
new k8s.KubeDeployment(this, 'Nginx', {  
  metadata: {  
    name: 'nginx'  
  },  
  spec: {  
    template: {  
      metadata: {  
        labels: { app: 'nginx' }  
      },  
      spec: {  
        containers: [{  
          name: 'nginx-main',  
          image: 'nginx'  
        }]  
      },  
    },  
    selector: {  
      matchLabels: { app: 'nginx' }  
    }  
  }  
});
```

Intent

```
new kplus.Deployment(this, 'Nginx', {  
  containers: [{ image: 'nginx' }]  
});
```

Now generally available! 🚀 🚀 🚀

Demo



cdk8s-plus: **Not** a mental model abstraction

```
const storageOptimizedNodes = kplus.Node.labeled(kplus.NodeLabelQuery.is('optimized', 'storage'));
const memoryOptimizedNodes = kplus.Node.labeled(kplus.NodeLabelQuery.is('optimized', 'memory'));

const dbService = new kplus.Service(this, 'DBService', { ports: [{ port: 8080 }] });
const db = new kplus.StatefulSet(this, 'DB', {
  containers: [{ image: 'db', portNumber: 1212 },
    service: dbService,
  }];
db.scheduling.attract(storageOptimizedNodes);

const cache = new kplus.Deployment(this, 'Cache', {
  containers: [{ image: 'cache', portNumber: 5050, envVariables: {
    DB_SERVICE: kplus.EnvValue.fromValue(dbService.name),
  } }],
  replicas: 2,
  spread: true,
});
cache.scheduling.attract(memoryOptimizedNodes);
const cacheService = cache.exposeViaService();

const web = new kplus.Deployment(this, 'Web', {
  containers: [{ image: 'web', portNumber: 9090, envVariables: {
    CACHE_SERVICE: kplus.EnvValue.fromValue(cacheService.name),
  } }],
  replicas: 2,
  spread: true,
});
web.scheduling.attract(memoryOptimizedNodes);
web.exposeViaService({ serviceType: kplus.ServiceType.LOAD_BALANCER });

cache.connections.allowFrom(web);
db.connections.allowFrom(cache);

web.scheduling.colocate(cache);

const frontOffice = kplus.Group.fromName(this, 'FrontGroup', 'frontoffice');

web.permissions.grantReadWrite(frontOffice)
cache.permissions.grantReadWrite(frontOffice)
db.permissions.grantReadWrite(frontOffice)
```

```
new WebApp(this, 'WebApp', {
  replicas: 5,
  colocated: true,
});
```

Protecting your manifests

“Human error continues to lead the causes of data breaches. A recent study revealed that human error was a major contributing factor in 95% of breaches.”

State of Kubernetes Security 2022, RedHat

✖ Ensure each container has a configured memory limit [1 occurrence]
Missing property object 'limits.memory' - value should be within accepted boundaries recommended by the organization

✖ Ensure workload has valid label values [1 occurrence]
Incorrect value for key(s) under 'labels' - the vales syntax is not valid so it will not be accepted by the Kubernetes engine

```

| Enable rules in policy "Staging" | 19
| Configs tested against policy    | 1
| Total rules evaluated            | 19
| Total rules failed               | 2
| Total rules passed              | 17
| See all rules in policy         | https://app.datree.io/login?cliId=rXG3u5t

```

```
K8S_8: "Liveness Probe Should be Configured"
LED for resource: Pod.default.pod1
File: /rootContainersFAILED.yaml:2-10
Guide: https://docs.bridgecrew.io/docs/bc\_k8s\_7
```

```
2 | apiVersion: v1
3 | kind: Pod
4 | metadata:
5 |   name: pod1
6 | spec:
7 |   containers:
8 |   - name: main
9 |     image: alpine
10 |    command: ["/bin/sleep", "999999"]
```

could be running with outdated image

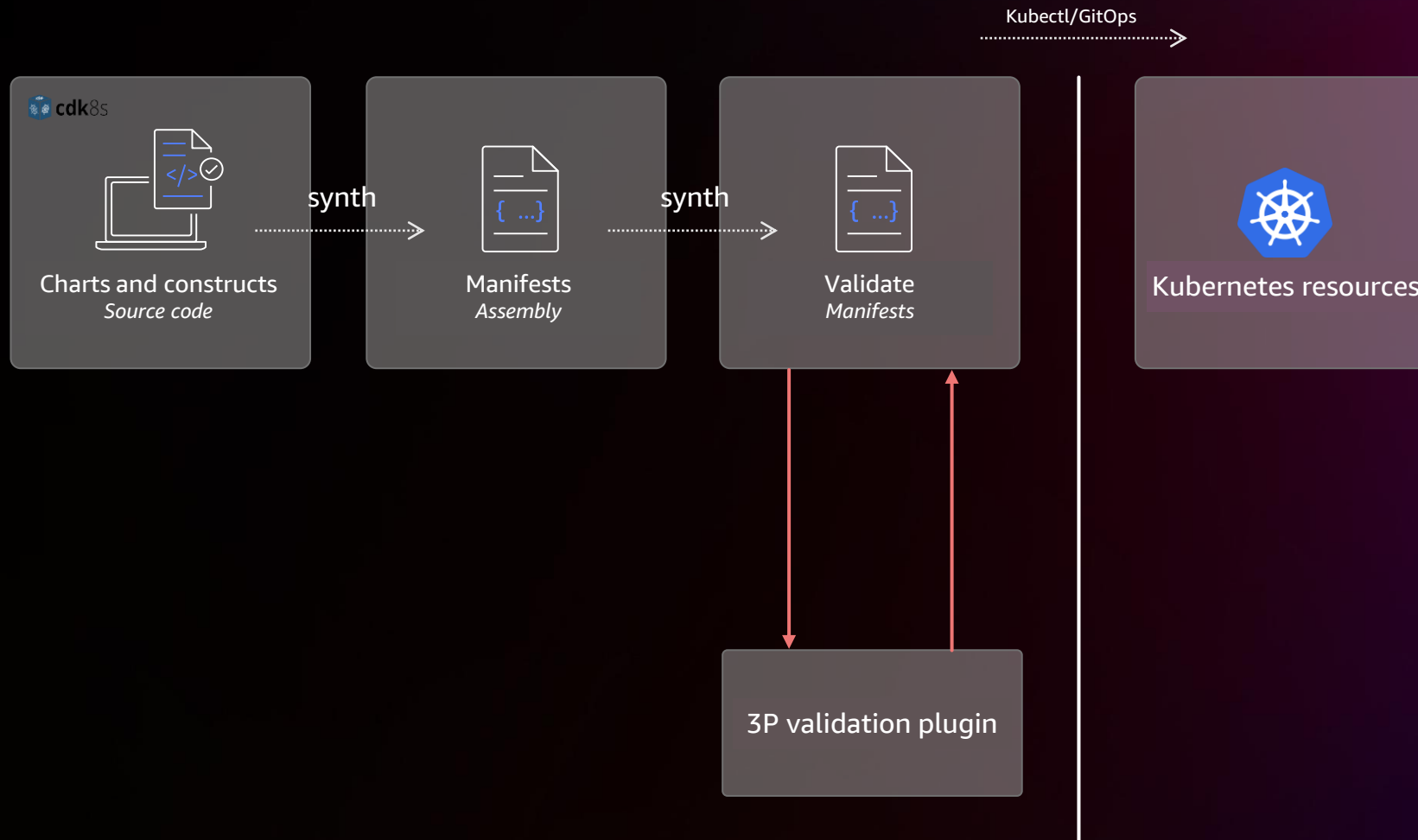
- ▼ **The issue is...**
The image policy does not prevent image reuse
- ▼ **The impact of this is...**
The container may run with outdated or unauthorized image
- ▼ **You can resolve this by...**
Set `imagePullPolicy` attribute to `Always`

```

16 metadata:
17   name: goof-mongo
18 spec:
19   replicas: 1
20   strategy: {}
21   selector:
22     matchLabels:
23       app: goof-mongo
24   template:
25     metadata:
26       labels:
27         app: goof-mongo
28   spec:
29     containers:
30     - image: mongo
31       name: goof-mongo
32       ports:
33       - containerPort: 27017
34       resources: {}

```

Protecting your manifests





Prevent misconfigurations in Kubernetes from reaching production

```
→ ~ datree test .datree/k8s-demo.yaml
>> File: .datree/k8s-demo.yaml

[V] YAML validation
[V] Kubernetes schema validation

[X] Policy check

✗ Ensure each container image has a pinned (tag) version [1 occurrence]
  - metadata.name: rss-site (kind: Deployment)
  ⚠ Incorrect value for key `image` - specify an image version to avoid
  unpleasant "version surprises" in the future

✗ Ensure each container has a configured memory limit [1 occurrence]
  - metadata.name: rss-site (kind: Deployment)
  ⚠ Missing property object `limits.memory` - value should be within the
  accepted boundaries recommended by the organization
```

👁 Watch 36 ▾

🍴 Fork 288 ▾

★ Star 6k ▾



Additional resources



cdk8s website



cdk8s plus documentation



Datree cdk8s integration repository



Demo code

Thank you!

Eli Polonsky

GitHub: @iliapolo

Twitter: @iliapolo

David Feldstein

GitHub: @davidfeldi

Twitter: @davidfeldi



Please complete the session survey in the **mobile app**



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.